

Prompt 1 — Arranque: validación crítica + colector mínimo

Urgente. Este prompt existe porque el dato de precios es perecedero: la tarifa de hoy para el 12 de octubre no se recupera mañana. Todo día sin captura es una curva de anticipación que nunca vamos a tener.

```
Vas a trabajar en el ecosistema de datos turísticos de Esquel (VPS Debian).

CONTEXT0 Y ESPECIFICACIÓN
El diseño completo está en el repo github.com/leanchoi/leanchoi, rama
claude/esquel-data-ecosystem-integration-qbxuv2. Cloná ese repo y leé, EN ESTE ORDEN:
1. AGENTS.md                – reglas duras. Las invariantes I1..I14 no se negocian.
2. docs/00-revision-critica.md – por qué el diseño es como es.
3. docs/01-motor-aereos.md   – motor de captura y cadencia.
4. docs/07-conectividad-sostenible.md §6 – por qué esta tarea es urgente.

No rediseñes. Las decisiones cerradas están en AGENTS.md §6 con su fundamento.

SISTEMAS EN EL VPS
/root/apps/tourism-intelligence-dashboard  Esquel DATA (:38520). NO lo toques todavía.
/root/scrapper/metrica                    Métrica (:3013). NO lo toques NUNCA (I1).
/opt/metrica-aereos                        lo vas a crear vos.
Chromium y Playwright ya están instalados y en uso por Métrica. Jamás corras
`playwright install` (I6).

=====
PARTE A – Dos pruebas que deciden el motor (horas, no días)
=====

Ejecutá specs/scripts/f0_validacion.py --check 1 3 en el VPS.
Instalá antes, en un venv propio: pip install fast-flights

F0-1 ¿Google Flights lista Flybondi y JetSMART en BUE→BRC?
Es LA prueba. Si no los lista, el motor mide solo Aerolíneas y toda comparación
Esquel vs Bariloche queda sesgada en la dirección que invalida la tesis del
proyecto.
• Si aparecen ambos -> seguí con el plan tal cual.
• Si falta alguno -> agregá scraping semanal de flybondi.com SOLO para BUE-BRC
como corrector de nivel. NO uses Amadeus (AGENTS.md §6).
• Verificación manual equivalente: abrí Google Flights con hl=es-AR&gl=AR&curr=ARS
y buscá Buenos Aires → Bariloche a 30 días.
```

- F0-3 30 consultas espaciadas 15-45 s desde la IP del VPS: ¿hay bloqueos?
- 0 errores -> el presupuesto de docs/01 §4.3 (~160 consultas/día) es válido.
 - 1-2 errores -> subí el espaciado a 60 s y recalculé el presupuesto.
 - 3 o más -> paré y reporté. Evaluamos SerpApi como motor primario.

El script fue especificado, no ejecutado contra servicios reales. La API de la versión de fast-flights que instales puede diferir: leé la firma real y adaptá `\_buscar()`. Si la adaptás, commiteá la corrección al script.

REPORTÁ EL RESULTADO DE A ANTES DE SEGUIR CON B.

=====

PARTE B – Colector mínimo en producción (Fla)

=====

Objetivo: que EMPIECE A EXISTIR EL HISTÓRICO. No que esté prolijo. La calidad se mejora después reprocesando desde la capa bronce, que existe exactamente para eso.

ALCANCE – solo esto:

- Rutas: BUE→EQS, COR→EQS, BUE→BRC, BUE→CPC, COR→BRC (tiers 1 y 2)
- Fechas: solo el conjunto "ancla" de specs/config/rutas_muestreo.json
- Salida: JSONL.gz crudo por corrida + bitácora de corridas
- Programación: systemd timer, 02:00-05:00 ART, concurrencia 1

FUERA DE ALCANCE en esta tarea: compactación a Parquet, tablas oro, frontend, modelos, fallback SerpApi, sonda Playwright.

IMPLEMENTACIÓN

/opt/metrica-aereos/

- | | |
|-----------------|---|
| aereos/tfs.py | Codificador protobuf del parámetro tfs. VENDORIZALO: copió y adaptó el encoder, no dependas de la librería en runtime. El esquema es chico y estable; la dependencia externa es el riesgo. |
| aereos/parse.py | Parser propio. VERIFICADO EN EL SPIKE: la respuesta son arrays JSON anidados sin esquema, NO HTML. Requisitos, derivados del bug de Flybondi que ya encontraste: <ul style="list-style-type: none">• Extracción por operador con FALLBACK: intentá la ruta conocida y, si falla, buscá el precio recorriendo la estructura (primer entero plausible en el rango esperado para la ruta) antes de declarar parse_error. Índices fijos a un array sin documentar son tan frágiles como parsear HTML.• Registrá en la bitácora POR QUÉ CAMINO se extrajo cada precio. Cuando Google cambie el layout, ese campo dice qué se rompió.• Fixtures con AL MENOS UN ITINERARIO DE CADA OPERADOR (AR, FO, WJ). Una fixture con solo Aerolíneas habría pasado el test que |

	no detectó el bug de Flybondi.
	• Test que asserta que los tres operadores se extraen.
aereos/schedule.py	Genera las consultas del conjunto ancla. Orden aleatorizado con prioridad por tier: si la corrida se trunca no debe perderse siempre la misma ruta.
aereos/collect.py	Orquestador. Espaciado uniform(15,45) s; pausa larga de 3-8 min cada 25 consultas; tope duro 250/día; circuit breaker a los 3 fallos consecutivos (aborta y marca el día como parcial).
aereos/runs.py	Bitácora. UNA FILA POR CONSULTA PLANIFICADA, salga bien o mal, con estado: ok sin_resultados bloqueado timeout parse_error omitido_por_presupuesto omitido_por_preflight. Más: itinerarios extraídos POR AEROLÍNEA y camino de extracción.
aereos/canario.py	Canario POR AEROLÍNEA, no del total. Alerta si los itinerarios de CUALQUIER operador caen >30% respecto de su propia mediana móvil de 7 días. Un contador global no habría detectado el fallo de Flybondi: los otros dos compensan, y la serie queda sesgada en silencio justo hacia donde invalida la tesis.
bin/preflight.sh	Aborta con código 0 (diferida, no fallida) si hay poca memoria libre o si hay procesos Chromium activos (`pgrep -c chromium`).

Esquema de los registros: specs/sql/01_air_schema.sql es el CONTRATO LÓGICO. Respetá campos, tipos y restricciones. NO lo ejecutes en PostgreSQL: la salida es JSONL. Prestá atención especial a return_date, trip_type y pax_count – sin ellos el TTCI no se puede calcular más adelante.

DOS COSAS QUE EL SCRIPT F0 HACE DISTINTO Y NO HAY QUE HEREDAR:

- El script consulta ONE-WAY. El colector debe consultar ROUND TRIP en las fechas ancla (return_offset_dias en rutas_muestreo.json). El TTCI se define sobre ida y vuelta; una serie one-way no sirve para el indicador central.
- Guardá TODOS los itinerarios, no solo el más barato. En la captura del spike, la misma ruta y fecha tenía vuelos a 295.935 y 547.402 ARS: esa dispersión de 1,85x es señal de clases tarifarias agotándose, que es el sensor de ocupación de docs/07 §2.2. Quedarse con el mínimo tira justo la información más valiosa.

RETENCIÓN DE CRUDO: la spec asumía HTML de ~40 KB por consulta y fijaba 14 días. Como la respuesta es JSON y pesa mucho menos, subí la retención a 90 días. Con el parser recién estrenado, poder reprocesar tres meses vale más que el disco que ocupa. Medí el tamaño real en la primera semana y ajustá.

systemd: Type=oneshot, Nice=10, MemoryMax=512M, CPUQuota=50%, ExecStartPre=preflight.sh. Timer, no un proceso residente con scheduler: no consume memoria mientras no corre.

Retención de HTML crudo: rotación de 14 días + 5 muestras diarias permanentes como fixtures. La retención completa serían ~2,4 GB/año y no se justifica.

LO QUE NO PODÉS HACER

- Tocar Métrica de ninguna forma (I1).
- Levantar contenedores o puertos nuevos (I5).
- Descargar navegadores (I6).
- Rellenar huecos de captura con datos interpolados (I12). Un hueco es un dato.
- Usar el motor sin registrar la corrida en la bitácora: sin eso es imposible distinguir "no había vuelo" de "no se pudo medir".

CRITERIOS DE ACEPTACIÓN

1. Tres noches consecutivas con cobertura $\geq 90\%$ de lo planificado.
2. RSS máximo del proceso < 200 MB (verificá con `systemd-cgtop`). Sin navegador debería estar en 60-90 MB; si se va a 400 MB estás levantando Chromium sin querer.
3. Duración de corrida < 120 min.
4. La bitácora registra TODAS las consultas planificadas, incluidas las fallidas.
5. Ninguna corrida solapa con la ventana de scraping de Métrica; sin degradación medible del VPS.
6. Tests de contrato del parser en verde sobre las fixtures.
7. ``systemctl disable --now metrica-aereos.timer`` y borrar `/opt/metrica-aereos` deja el VPS exactamente como estaba (I7).

ENTREGABLE FINAL

Un reporte con: resultado de F0-1 y F0-3, qué supuestos de la spec resultaron falsos y cómo los corregiste (en el mismo commit), y las tres noches de cobertura medida.